Manuel-Complet-OpenVisio

OpenVisio v40.2 — édition documentaire cumulative
Auteur : Abdelhalim Talbi
État : exemples fictifs identifiés ; qualification native Windows non réalisée.

Périmètre et traçabilité

Cette documentation décrit les sources disponibles dans l'archive cumulative. La version applicative demeure 40.1.0 ; la version documentaire et de packaging est 40.2.

1. Guide de prise en main

Guide détaillé dans documentation/01-LIRE-EN-PREMIER

2. Objectifs

OpenVisio est un éditeur de diagrammes orienté infrastructures, SAN et documentation technique. Cette livraison v40.2 porte sur la qualité documentaire ; le moteur applicatif reste v40.1.

3. Démarrage guidé

Ouvrir l'application ; utiliser Démarrer ; choisir Nouveau, Ouvrir ou un modèle ; déposer des formes sur le canevas ; relier les équipements ; renseigner les propriétés ; enregistrer.

4. Scénario illustratif

EXEMPLE FICTIF : document « SAN-Site-A » avec deux fabrics indépendantes, deux commutateurs, quatre serveurs et deux baies. Vérifier visuellement les connexions et renseigner les WWPN.

5. État de validation

Tests Node.js à rejouer dans le pipeline ; aucune compilation native ni capture Windows certifiée disponible dans cet environnement.

6. Guide utilisateur détaillé

Guide détaillé dans documentation/02-UTILISATION

7. Créer et éditer

Nouveau diagramme : Ctrl+N hors zone de saisie. Ouvrir : Ctrl+O. Enregistrer : Ctrl+S. Rechercher une forme : touche / hors dialogue. La bibliothèque propose favoris, récentes et catégories métier.

8. Inspecteur

Sélectionner un objet ; filtrer les champs via Général, Apparence, Connexions, SAN ou Résilience. La recherche permet de retrouver les propriétés avancées.

9. Sauvegarde

La sauvegarde sur disque utilise File System Access si disponible ; la validation attend la fermeture du flux. Le téléchargement JSON classique reste possible, mais ne garantit pas à lui seul l'écriture effective.

10. Cas fictif SAN

EXEMPLE FICTIF : switch FAB-A-SW01, fabric A, VSAN 10 ; switch FAB-B-SW01, fabric B, VSAN 20 ; deux chemins distincts pour un hôte. Ces noms et valeurs ne sont pas issus d'un environnement réel.

11. Procédure de recette

Créer un diagramme, ajouter deux équipements, sauvegarder, rouvrir, contrôler noms/connexions, modifier une propriété, vérifier l'indicateur de modifications.

12. Guide d'installation Windows

Guide détaillé dans documentation/03-INSTALLATION

13. Prérequis

Windows 10/11 compatible, runtime WebView2, droits suffisants pour installer, package portable ou installateur issu d'une compilation Windows qualifiée.

14. Procédure

Décompresser l'archive cumulative ; lire README.md ; ouvrir application/windows ; lancer le préflight PowerShell ; générer les binaires selon le guide compilation ; effectuer la recette sur une machine Windows.

15. Commande préflight

PowerShell : powershell -ExecutionPolicy Bypass -File .\windows\Preflight-Windows.ps1

SORTIE ATTENDUE (EXEMPLE FICTIF) : PASS | .NET SDK ; PASS | WebView2 ; PASS | Inno Setup.

16. Dépannage

Si WebView2 manque : installer le runtime officiel. Si dotnet introuvable : installer le SDK prévu et relancer le terminal. Si l'installateur échoue : conserver journaux et code retour.

17. Limite

Aucun EXE Windows réellement compilé n'est livré dans cette archive de sources.

18. Guide de compilation et de packaging

Guide détaillé dans documentation/04-COMPILATION

19. Préflight

Depuis application/ sous Windows : powershell -ExecutionPolicy Bypass -File .\windows\Preflight-Windows.ps1

20. Publication .NET

EXEMPLE DE COMMANDE À ADAPTER : dotnet publish .\windows\native\OpenVisio.csproj -c Release -r win-x64 --self-contained false

SORTIE ATTENDUE (FICTIVE) : Build succeeded ; répertoire bin\Release\...\publish.

21. Portable

Exécuter .\windows\Build-Portable-Windows.ps1 après validation des prérequis ; vérifier la présence des ressources HTML, JS et CSS.

22. Installateur

Examiner windows/installer/OpenVisio.iss ; compiler avec Inno Setup si présent ; vérifier signature, désinstallation, lancement et mise à jour.

23. Tests

node --test tests/*.test.mjs ; python prepare-release.py --output OpenVisio-v40.2.zip. La publication bloque si les tests échouent.

24. Résultats attendus

Les exemples de console sont des sorties simulées pour documentation, pas des résultats d'exécution sur Windows.

25. Architecture technique détaillée

Guide détaillé dans documentation/05-ARCHITECTURE

26. Composants

Hôte natif Windows en C# / WinForms ; WebView2 ; interface HTML/CSS ; modules JavaScript ; modèle de diagramme ; analyse SAN ; export JSON et SVG.

27. Flux de données

Événement utilisateur → contrôleur UI → mutation du modèle → rafraîchissement du rendu → état dirty → sérialisation/export.

28. Sécurité et résilience

Vérifier les entrées JSON ; ne pas considérer un téléchargement comme écriture confirmée ; conserver les fichiers locaux hors application ; isoler les exemples fictifs des données de production.

29. Interfaces

Les modules JavaScript internes ne constituent pas une API HTTP publique. Le serveur local éventuellement utilisé pour servir les ressources ne doit pas être présenté comme une API métier sans route effectivement implémentée.

30. Exemple de JSON fictif

{ "name": "SAN-Site-A", "devices": ["FAB-A-SW01", "FAB-B-SW01"] }

ATTENTION : schéma conceptuel simplifié, non garanti compatible avec le format d'import OpenVisio.

31. Limitations

L'intégration et la sauvegarde native WebView2 nécessitent une recette Windows réelle.

32. Guide de qualification et exemples de résultats

Guide détaillé dans documentation/06-QUALIFICATION

33. Tests automatisés

Depuis application : node --test tests/*.test.mjs

SORTIE ATTENDUE (FICTIVE) : # pass 1661 ; # fail 0. Le nombre réel peut évoluer.

34. Recette UI

Vérifier démarrage, nouveau, ouverture, bibliothèque, recherche, inspecteur, connexions, undo/redo, sauvegarde, import, export SVG, fenêtres modales.

35. Recette SAN

Créer deux fabrics indépendantes, simuler une perte de chemin, vérifier le diagnostic et la présence des informations de résilience.

36. Recette CLI

python prepare-release.py --output OpenVisio-v40.2.zip

SORTIE ATTENDUE (FICTIVE) : PASS version=40.2 files=... ; code retour 0.

37. API

Aucune API métier HTTP publique certifiée pour v40.2. Ne pas inventer d'URL ni de réponse API ; les exemples de commandes documentent seulement les interfaces réellement présentes.

38. Critères d'acceptation

Aucun test échoué ; ZIP CRC valide ; empreintes concordantes ; Word/PDF lisibles ; capture réelle identifiable par date, version et contexte ; installation Windows qualifiée séparément.

39. Inventaire des écrans et protocole de capture

Guide détaillé dans documentation/07-CAPTURES

40. Statut réel

Aucune capture native Windows/WebView2 v40.2 n'a été réalisée dans cet environnement. Les diagrammes fournis sont des illustrations techniques, non des captures.

41. Écrans à capturer

Accueil ; éditeur vide ; bibliothèque Toutes/Favoris/Récentes ; recherche ; canevas avec diagramme ; inspecteur Général/Apparence/Connexions/SAN/Résilience ; dialogues ouvrir/enregistrer ; export SVG ; outils SAN ; confirmations et erreurs.

42. Protocole

Sur Windows : noter version, résolution, date, scénario ; capturer la fenêtre entière ; masquer les secrets ; conserver PNG original ; joindre un fichier de provenance et un résultat de recette.

43. Captures CLI

Capturer la commande réellement exécutée, son code de retour et sa sortie. Ne jamais réutiliser une sortie fictive comme preuve de test.

44. Exemples

Les organigrammes 01 à 05 du dossier 08-ILLUSTRATIONS sont des schémas conceptuels explicitement étiquetés.

Organigrammes et illustrations

01 Parcours utilisateur



Schéma conceptuel — aucune capture d'interface réelle.

02 Architecture

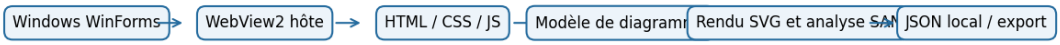


Schéma conceptuel — aucune capture d'interface réelle.

03 Compilation



Schéma conceptuel — aucune capture d'interface réelle.

04 Sauvegarde



Schéma conceptuel — aucune capture d'interface réelle.

05 Qualification



Schéma conceptuel — aucune capture d'interface réelle.

Points de contrôle

- Vérifier les chemins, scripts et prérequis avant exécution.
- Conserver le code retour, les journaux et les résultats réellement observés.
- Distinguer résultats attendus fictifs et résultats de qualification réels.
- Compléter les captures sur une machine Windows avec WebView2.